Expert-Notes: Vom 1D-Kalman-Filter zum ESKF

Projekt: ModelFilterIBVS

Dokumentation der mathematischen Herleitung

22. Februar 2026

1 Einführung: Das 1D-Kinematik-Beispiel

Ein greifbares Beispiel ist der beste Weg, um die abstrakte Mathematik des Kalman-Filters zu entmystifizieren. Wir betrachten einen Roboter auf einer 1D-Schiene. Unser **Zustandsvektor** x_k besteht aus Position p und Geschwindigkeit v:

$$x_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}$$

Der Filter speichert diesen Zustand und die **Fehlerkovarianzmatrix** P_k (Unsicherheit).

1.1 Schritt 1: Die Prädiktion (Vorhersage)

Wir nutzen unser physikalisches Modell, um „blind“ in die Zukunft zu rechnen:

1. **Zustandsextrapolation:** $x_{k|k-1} = Fx_{k-1|k-1} + Bu_k$
   Bei konstanter Geschwindigkeit für ein Intervall Δt ist $F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$. Bu_k (Steuerung) setzen wir hier auf 0.
2. **Kovarianzextrapolation:** $P_{k|k-1} = FP_{k-1|k-1}F^T + Q$
   Die Unsicherheit wächst durch das Prozessrauschen Q (z. B. Wind).

1.2 Schritt 2: Die Korrektur (Update)

Fusion der Vorhersage mit einer Sensormessung z_k:

1. **Kalman-Gain:** $K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$
   Dynamischer Gewichtungsfaktor: Vertraue ich dem Modell (P) oder dem Sensor (R)?
2. **Zustandsupdate:** $x_{k|k} = x_{k|k-1} + K_k(z_k - Hx_{k|k-1})$
   Die *Innovation* (Differenz Messung zu Erwartung) korrigiert die Vorhersage.
3. **Kovarianzupdate:** $P_{k|k} = (I - K_kH)P_{k|k-1}$

2 Erweiterung auf n Dimensionen

Der Standard Kalman-Filter funktioniert problemlos auch im 3D- oder n-dimensionalen Raum. Die Dimensionalität ändert am Ablauf nichts; die Vektoren werden lediglich länger. Im **3D-Raum** (x, y, z) gilt:

$$x_k = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^T$$

Matrizen wie F und P wachsen entsprechend auf 6×6.

3 Die Grenzen der Linearität im Projekt-Kontext

Obwohl der Filter 3D-Positionen gut schätzt, stößt er bei der Schätzung einer **Homographie-Matrix** $H \in \mathbb{R}^{3\times3}$ an eine harte Grenze.

3.1 Das Problem der projektiven Geometrie

In der projektiven Geometrie gilt $p(t) \propto H(t)p^*$. Das bedeutet: H und ein Vielfaches (z. B. $2H$) beschreiben dieselbe Transformation. Um Eindeutigkeit für den Filter zu schaffen, normalisieren wir die Matrix so, dass gilt:

$$\det(H) = 1$$

Damit liegt H in der **Speziellen Linearen Gruppe** $SL(3)$.

3.2 Warum der Standard-Filter hier versagt

Eine Lie-Gruppe wie $SL(3)$ ist kein flacher Vektorraum, sondern eine gekrümmte **Mannigfaltigkeit**. Die lineare Addition ist hier nicht abgeschlossen.

Wenn $A, B \in SL(3)$, dann gilt meistens:

$$\det(A + B) \neq 1$$

Nutzt man das Standard-Update $H_{neu} = H_{alt} + \Delta H$, wird die Gruppenstruktur zerstört. Die Matrix verliert ihre geometrische Bedeutung.

4 Die Lösung: Error-State Kalman Filter (ESKF)

Da wir nicht addieren $(+)$ dürfen, müssen wir multiplizieren $(\cdot)$. Die Dynamik wird auf dem Tangentialraum der Gruppe, der **Lie-Algebra** $\mathfrak{sl}(3)$, modelliert.

Der ESKF trennt den Zustand elegant:

- **Nominaler Zustand:** Die Homographie auf der gekrümmten $SL(3)$-Gruppe.
- **Error-State (Fehlerzustand):** Ein lokales Update auf dem flachen Tangentialraum $\mathfrak{sl}(3)$, auf dem wir wieder linear rechnen können.

Dieses Prinzip ist für das Projekt **ModelFilterIBVS** essenziell, um Bild-Ausfälle (Blendung) durch die Roboterkinematik stabil zu überbrücken.